DynaExQ: Budget-Safe Online Precision Residency for Single-GPU MoE Inference

Kexin Chu, Dawei Xiang, and Wei Zhang

***Abstract*—Large Mixture-of-Experts (MoE) models are difficult to serve on one GPU because all expert weights must remain addressable although each token activates only a small subset. Offloading reduces resident memory but can expose PCIe transfers when batched routing produces a dense expert working set. Static post-training quantization fixes precision offline and cannot adapt its allocation to routing shifts. DynaExQ instead treats expert precision residency as online resource allocation under an explicit expert-memory budget. A routing-driven policy selects high-traffic experts, while an admission-controlled transition mechanism copies prepared precision versions on a separate CUDA stream and publishes a new expert handle only after transfer completion. Partitioned fixed-block pools bound the resident and in-flight transition memory owned by the runtime; end-to-end safety additionally requires the serving stack to honor its declared KV-cache and activation reservations. Across five accuracy benchmarks on three MoE models, DynaExQ recovers 77–79% of each model's quality gap between its uniform low- and high-precision references. Performance experiments compare fully resident execution against uniform PTQ on all models and against a pinned open-source MoE-Infinity runtime on its supported Qwen3-30B checkpoint. Component studies quantify the effects of online selection, asynchronous transitions, hysteresis, and the high-precision budget.**

***Index Terms*—Mixture-of-Experts, large language model inference, GPU memory management, mixed-precision quantization, runtime systems**

I. Introduction

Mixture-of-Experts (MoE) models [1] scale neural-network capacity by routing each token to a small subset of expert subnetworks, keeping per-token compute bounded while the full parameter set must stay accessible for unconstrained routing. Modern MoE language models such as Mixtral [2], DeepSeek-V2 [3], and Qwen3 [4] contain tens to hundreds of billions of parameters, yet activate only a few billion per token. Deploying these models on a single high-end GPU exposes a fundamental hardware constraint: device memory must simultaneously hold expert weights, non-expert parameters, activations, the KV cache, and runtime buffers. Qwen3-Next-80B, for instance, requires roughly 160 GB in FP16, far exceeding the device-memory capacity of the 48 GB GPU used in our evaluation. The deployment bottleneck is therefore not peak arithmetic throughput but *on-device memory management*: deciding which expert weights remain resident, at what precision, and how residency adapts as the workload changes.

Two families of techniques address this memory-capacity wall. The first, expert offloading and prefetching [5], [6], [7], [8], [9], [10], [11], [12], [13], treats GPU memory as a managed cache and moves experts across the PCIe memory hierarchy on demand. The second, post-training quantization (PTQ) [14], [15], [16], [17], compresses expert weights to low bit widths, reducing the per-expert footprint so the full pool fits in device memory.

Offloading works well when each iteration touches a small, stable expert subset. In practice, MoE activation becomes substantially denser during prefill and at larger batch sizes (Table I), expanding the per-iteration working set beyond what can be staged within the PCIe compute-transfer overlap window. Once that window is exceeded, data movement stalls the GPU compute pipeline and amplifies worst-case latency (Figure 1).

PTQ avoids cross-tier data movement entirely and, with no ongoing migration, achieves the lowest raw per-step latency. However, most PTQ pipelines fix a uniform or per-expert precision map offline. Expert utilization is heavy-tailed and task-dependent: the identity of frequently activated experts shifts across text, math, and code workloads (Figure 2). A static precision map therefore wastes scarce device-memory capacity on experts that carry little traffic in the current task while over-compressing those that become critical. This paper does not claim to beat static PTQ on raw latency; it targets a better *accuracy–throughput–memory* tradeoff under the same fixed device-memory budget.

We propose DynaExQ, which manages expert precision as a dynamically allocated on-device resource for single-GPU MoE inference under a declared device-memory envelope. During inference, the system observes router outputs, estimates which experts carry disproportionate traffic over a stable time horizon, and allocates a limited high-precision budget to those experts. The remaining experts stay at lower precision. Preallocated pools and admission control keep all runtime-owned expert and transition memory within the envelope; the end-to-end cap holds when the deployment's KV-cache and activation reservations are respected.

The design separates policy from mechanism. A scheduler periodically selects budget-feasible high-precision sets per layer. A transition pipeline carries out promotions and demotions on a dedicated CUDA stream without explicit transition synchronization on the forward path. Pool-based memory management with admission control prevents the runtime-owned expert footprint from exceeding its budget. Section III describes each component.

The main contributions of this work are:

- We formulate single-GPU MoE inference under a hard device-memory constraint as an on-device precision residency management problem, targeting the accuracy–

The authors are with the University of Connecticut, Storrs, CT 06269 USA (e-mail: kexin.chu@uconn.edu; ieb24002@uconn.edu; wei.13.zhang@uconn.edu). Corresponding author: Wei Zhang.

throughput–memory tradeoff rather than minimum raw latency alone.

- We design an asynchronous runtime execution model with versioned expert handles. A new handle is published only after its copy completes, and read leases plus per-compute-stream last-use events protect the old block until every dispatch and kernel that can reference it has drained.
- We introduce deterministic, pool-based device-memory management with admission control. Exact expert-sized blocks eliminate variable-size external fragmentation, per-tier capacities isolate resident slots, and a budget tracker prevents transient over-commitment during concurrent migrations.
- We evaluate DynaExQ on Qwen3-30B, Qwen3-Next-80B, and Phi-3.5-MoE using five accuracy benchmarks plus WikiText-2 perplexity. Across the three models, DynaExQ recovers 77–79% of the model-specific low-to-high-precision quality gap; on Qwen3-Next-80B, for example, it raises the average from 73.58% to 77.15% toward the 78.12% INT4 reference. Performance experiments use uniform PTQ on all models and a pinned open-source MoE-Infinity runtime where that implementation supports the evaluated checkpoint; ablations isolate each mechanism, and raw peak-memory traces test the 48 GB deployment envelope.

II. Background and Motivation

A. MoE Inference on a Single GPU

On a single accelerator, expert weights dominate the memory footprint; they comprise roughly 95–96% of the parameters in the three models studied here [4], [18], [19]. Whether a model fits in device memory depends on how those weights are stored. Expert utilization is also inherently uneven: a small hot set receives a disproportionate share of traffic, so allocating equal fidelity to every expert is rarely cost-effective under a tight memory envelope.

B. Offloading and Prefetching

A family of systems treats GPU memory as a managed cache and fetches experts on demand [5], [6], [7], [8], [9], [10], [11], [12], [20], [13]. These methods work best when each iteration activates a small, stable working set. We measure density as the number of unique selected experts per MoE layer, first over all nonpadding tokens in a 2,048-token-capped prefill and then over the immediately following one-token decode. Each table cell averages five deterministic prompt blocks and all routed layers; batch-b uses the first b prompts of the same 32-prompt block. In practice, activation becomes substantially denser as batch size grows: for Qwen3-30B at batch 32, Table I shows 94.0% of experts selected during prefill and 26.0% in the next decode step. The distinction matters: accumulating selections over many decode steps would overstate the instantaneous working set. Figure 1 includes two directly measured Qwen blocking-load series and a measured Phi-3.5-MoE cold-cache routing-trace replay. Both measurements report time exposed by serial expert movement on an RTX A6000 rather than end-to-end behavior of an overlapping offload system.

TABLE I
Mean unique-expert activation ratio (%) across routed layers and five prompt blocks.

Model	Stage	bs=1	2	4	8	16	32
Qwen3-30B	Decode	6.3	7.9	11.8	16.1	20.8	26.0
	Prefill	59.8	70.0	82.9	89.7	92.4	94.0
Qwen3-Next-80B	Decode	1.9	3.7	6.7	14.9	25.4	39.2
	Prefill	24.6	35.3	48.8	67.1	78.9	86.2
Phi-3.5-MoE	Decode	12.5	22.7	37.1	59.3	77.8	88.8
	Prefill	98.2	98.8	99.4	99.8	99.9	100.0

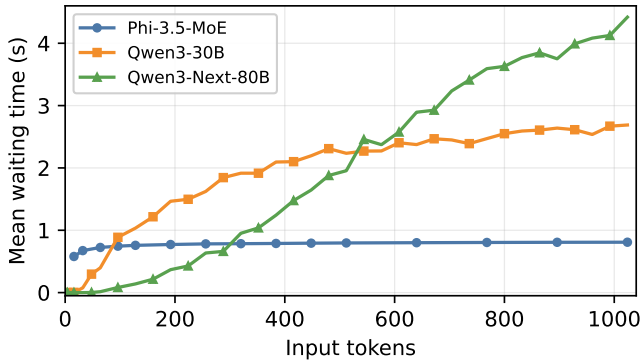


Fig. 1. Mean blocking expert-transfer waiting time on one RTX A6000. Qwen values are direct per-request measurements; Phi-3.5-MoE uses cold-cache routing-trace replay. Each point averages 10 measured requests or trials.

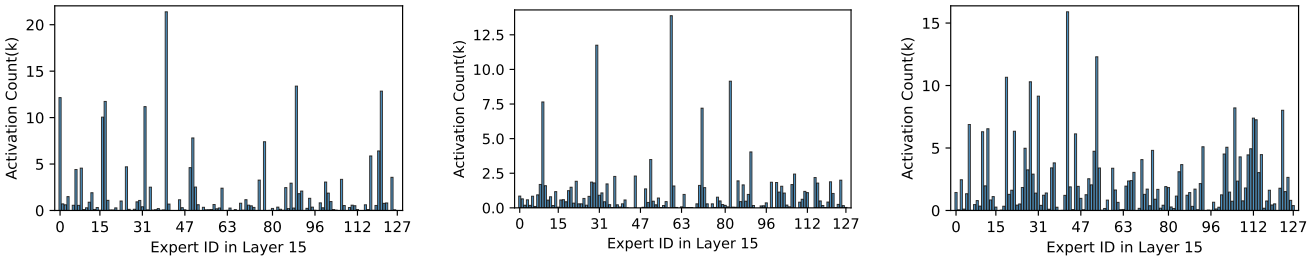
C. Static Post-Training Quantization

PTQ compresses weights to low bit widths without retraining. General-purpose schemes such as GPTQ [15], AWQ [14], SmoothQuant [21], and LLM.int8() [22] have been extended to MoE-specific settings [16], [17], [23], where routing-induced imbalance, biased calibration data, and heterogeneous execution require per-expert or per-block bit allocation. PTQ avoids cross-tier movement on the critical path and achieves the lowest raw per-step latency, but most pipelines fix a precision map offline, implicitly assuming expert importance is stable at inference time.

That assumption is brittle for MoE. Expert utilization is heavy-tailed and the identity of hot experts shifts across workloads (Figure 2). We count every selected top-k token–expert dispatch, rather than thresholding router probabilities, and reset the counter between the three full evaluation workloads. Under this pinned protocol, the top-10 most frequently selected experts for text, math, and code are entirely disjoint.

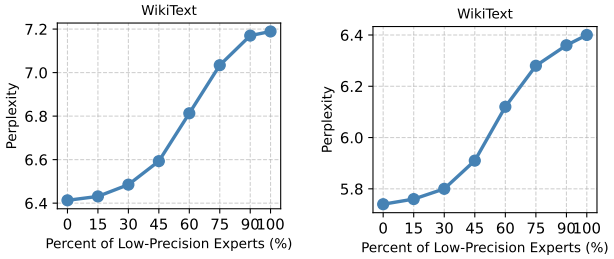
D. Motivation and Challenges for Online Precision Control

These results suggest adapting expert precision online under a strict GPU budget. A natural concern is whether such adaptation destabilizes quality. We concatenate the pinned WikiText-2 test split and measure perplexity over at most 128 exact 2,048-token windows while increasing the fraction of low-precision experts per layer. We obtain one expert ranking from the independent calibration corpus, freeze online reassignment, and demote an exact suffix of that same ranking at every point; thus, the sweep isolates the precision quota rather than conflating it with scheduler adaptation. As Figure 3 shows, when demotion targets infrequently activated experts first, perplexity rises smoothly.



(a) WikiText (b) GSM8K (c) HumanEval

Fig. 2. Selected token–expert dispatch counts at Layer 15 of Qwen3-30B under the full pinned WikiText, GSM8K, and HumanEval workloads. The scheduler is frozen with all experts at INT4; the top-10 most frequently selected experts are entirely disjoint across text, math, and code tasks.



(a) Qwen3-30B (b) Qwen3-Next-80B

Fig. 3. Perplexity vs. low-precision expert ratio (Qwen3-30B: FP16 vs. INT4; Qwen3-Next-80B: INT4 vs. INT2).

Realizing this tradeoff at runtime, however, requires satisfying three constraints:

- **Hard expert-budget invariance.** Precision changes alter the resident footprint. The runtime must never exceed the device envelope allocated to expert residency and transitions; otherwise out-of-memory failures or forced evictions cascade into instability. The deployment must separately size KV-cache and activation reserves for its supported request shape.
- **Critical-path isolation.** Precision transitions involve heavyweight device transfers that must not stall the forward pass.
- **Tail-latency robustness.** Naive reallocation can thrash when routing scores fluctuate, amplifying migration traffic and widening P99 latency.

III. DynaExQ Architecture

DynaExQ treats expert precision as an online, budget-constrained resource. Each expert has a low-precision representation (b^{lo}) and a high-precision representation (b^{hi}) in host memory, while exactly one version is active on the device. At runtime, the system adapts which experts occupy b^{hi} based on router-observed utilization, subject to three constraints: (C1) budget feasibility, (C2) no explicit transition synchronization on the forward path, and (C3) stability under routing fluctuations.

A. Architecture Overview and Execution Model

Figure 4 shows the end-to-end control loop. The design separates *policy* (what to change) from *mechanisms* (how to change safely).

a) Policy: The scheduler interprets router-derived traffic and chooses a budget-feasible high-precision resident set per layer: hotness estimation via periodic aggregation, top-n selection under fixed per-layer capacities, and hysteresis to damp oscillation. It emits target sets and promotion/demotion candidates but does not allocate GPU memory directly.

b) Mechanisms: Three mechanisms implement this runtime execution model: (i) versioned, representation-immutable per-expert handles with publish-after-complete semantics isolate dispatch from in-flight data movement; (ii) partitioned device pools and a budget tracker admit or reject each transition before any allocation; (iii) an asynchronous transition pipeline runs data movement on a dedicated CUDA stream and publishes a new handle only after transfer completes. Consequently, transition submission does not wait for a copy, dispatch cannot observe a partially populated buffer, and no transition starts without a prior reservation against the declared device-memory envelope. Stream separation does not eliminate memory-bandwidth contention, which we measure end to end.

c) Worker-side dispatch: On the worker side, each request invokes the router to obtain top-k expert IDs. Each ID resolves to a versioned handle containing a fully materialized device block and its quantization metadata. Dispatch observes either the previous complete handle or the next complete handle; it does not mutate a handle in place.

d) Scheduler-side control: The scheduler consumes router traces asynchronously. Per-expert hotness statistics feed a budget-feasible selection that determines, for each layer ℓ, a high-precision resident set subject to a layer capacity $n_{\text{hi},\ell}$. The resulting target sets drive a transition manager that maintains promotion/demotion queues and issues background work on a migration stream.

e) Budget initialization: Before inference begins, the system derives per-layer capacities $\{n_{\text{hi},\ell}\}$ from the total device memory M_{total}. Let M_{fixed} denote declared reservations for non-expert parameters, activations, KV cache, kernel-conversion workspace, and a safety margin. Let M_{trans} be the preallocated headroom for at most K simultaneous publish-before-reclaim transitions, and let m_ℓ^{hi} and m_ℓ^{lo} be the measured packed bytes of one expert at each precision tier in layer ℓ. The initialization chooses per-layer capacities such that:

$$\begin{aligned} M_{\text{resident}} &= \sum_{\ell} \left[n_{\text{hi},\ell} m_\ell^{\text{hi}} + (E_\ell - n_{\text{hi},\ell}) m_\ell^{\text{lo}} \right], \\ M_{\text{trans}} &= K \max_{\ell, b \in \{\text{hi},\text{lo}\}} m_\ell^b, \\ M_{\text{resident}} + M_{\text{trans}} &\le M_{\text{total}} - M_{\text{fixed}}. \end{aligned} \tag{1}$$

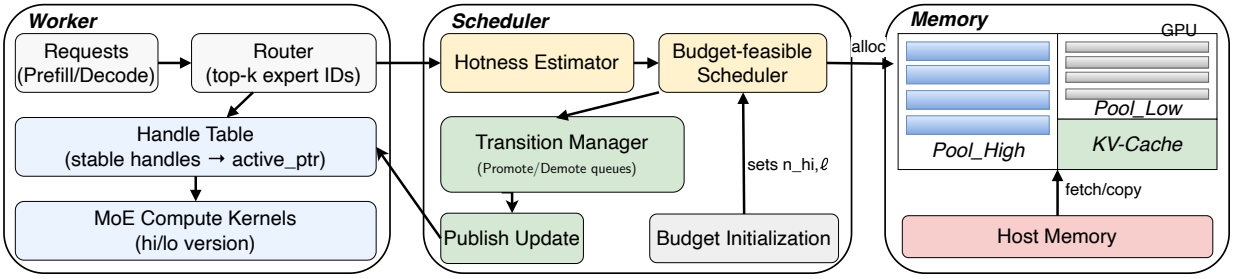


Fig. 4. System architecture of DynaExQ. The worker resolves a versioned expert handle; a scheduler observes router traces and updates precision residency under a strict device-memory budget; a transition pipeline carries out promotions and demotions on a dedicated migration stream.

where E_ℓ is the total number of experts in layer ℓ. Uniform, expert-count-proportional, and greedy capacity allocation are implemented; reported experiments use the deterministic proportional strategy. Before a reported run, an independent train/validation calibration trace executes the uniform all-low representation and ranks experts by arithmetic mean routed probability mass across prompts. Unlike the online EMA, this accumulator is order-invariant; it avoids both recency bias and seeding the ranking with an arbitrary mixed-precision identity set. Initialization takes the first $n_{\mathrm{hi},\ell}$ entries of that same pinned ranking, and different budget-sensitivity points change only the prefix length. The checkpoint, calibration-source hash, selected prompt identifiers, aggregation rule, ranking hash, and clean code revision are recorded with each result. An expert-ID-prefix fallback exists only for tests and exploratory runs and is rejected by the result audit. After initialization, $n_{\mathrm{hi},\ell}$ remains fixed; only expert *identities* change online.

f) Scope of the memory invariant: Equation (1) is enforced exactly for memory owned by DynaExQ: resident expert blocks, staging blocks, and conversion workspace are allocated or reserved at startup, and no transition is admitted beyond those capacities. The whole-process device-memory cap additionally assumes that KV cache, activation, and framework allocations remain within their declared reservations. Startup checks available GPU memory against all declared components, but the prototype does not intercept a framework allocator if a later request exceeds its KV or activation reserve. We report both the expert-pool counters and the observed process peak to distinguish the enforced invariant from this deployment assumption.

B. Online Scheduling Policy

The scheduling policy determines which experts should occupy the limited high-precision capacity. It is kept simple so that its behavior is easy to reason about and its per-tick cost stays small relative to inference.

a) Hotness estimation: For a forward step with N_ℓ routed token positions, let $p_{\ell,t,e}$ be the router probability for expert e when it is selected in the top-k set $\mathcal{R}_{\ell,t}$, and zero otherwise. The runtime uses the selected probability mass per token,

$$c_{\ell,e} = \frac{1}{N_\ell} \sum_{t=1}^{N_\ell} \mathbf{1}[e \in \mathcal{R}_{\ell,t}]\, p_{\ell,t,e}.$$

Thus the signal captures both selection frequency and router confidence. Updates are applied after every observation, with $c_{\ell,e} = 0$ for unselected experts so stale scores decay. Residency decisions are triggered every T_u forward steps; the step-based definition is deterministic under replay and makes the number of router observations per decision explicit. The smoothed hotness score is:

$$S_{\ell,e} \leftarrow \alpha \cdot S_{\ell,e} + (1-\alpha) \cdot c_{\ell,e},$$

where $\alpha \in [0,1)$ controls how quickly past observations decay. This construction uses router outputs only and requires no labels or online quality signals.

b) Budget-feasible selection: For each layer ℓ with capacity $n_{\mathrm{hi},\ell}$, the high-precision set is:

$$H_\ell \leftarrow \mathrm{TopN}(\{S_{\ell,e}\}_e,\ n_{\mathrm{hi},\ell})\,.$$

Selection is local to each layer and budget-feasible by construction: no cross-layer coordination is needed. The set difference against the current resident set yields promotion candidates $H_\ell \setminus H_\ell^{\mathrm{cur}}$ and demotion candidates $H_\ell^{\mathrm{cur}} \setminus H_\ell$. These candidates are passed to the transition pipeline, which admits promotions only when the budget tracker can reserve the required memory (Section III-C).

c) Hysteresis for stability: A plain top-N rule can cause unnecessary churn when several experts have similar hotness scores. Each swap triggers a promotion and a matching demotion, consuming migration bandwidth without improving quality. We therefore apply additive-margin hysteresis: after the cold-start set reaches capacity, the highest-scoring outsider replaces the lowest-scoring incumbent only when their score difference exceeds δ. Candidate pairs are processed in score order until the inequality fails. Hysteresis does not change the budget constraint; it reduces oscillations and makes the transition rate more predictable under transient routing fluctuations.

The policy produces a target set per layer at cadence T_u. A target does not become visible merely because it was selected: each expert dispatch continues to acquire the last published handle until that expert's pending version switch completes.

C. GPU Memory Management

Dynamic residency stresses GPU memory in two ways. First, promotions and demotions populate destination representations in large weight buffers; if these buffers share a general-purpose device heap with variable request sizes, external fragmentation and allocator jitter can inflate latency variance. Second, several transitions may overlap in time; if their aggregate footprint is unbounded, concurrent promotions can exhaust the device budget even when each is individually feasible.

a) Partitioned pools and fragmentation control: We divide the GPU memory assigned to expert weights into disjoint per-layer, per-tier pools: `pool_hi` for high-precision versions and `pool_lo` for low-precision versions. Their capacities are fixed independently of the logically reserved KV-cache and framework budgets. Within each pool, one fixed-size byte block stores one expert's measured packed representation. This eliminates variable-size external fragmentation and keeps the free/allocate path uniform, following the same principle as paged KV-cache memory in LLM inference engines [24]. Each pool maintains its own free list, so transitions never invoke the general-purpose device allocator. Separating capacities by precision tier prevents churn in one tier from consuming blocks assigned to the other. Rapid promotions filling `pool_hi`, for example, cannot encroach on `pool_lo`'s resident set.

b) Budget model and admission: A `BudgetTracker` enforces the global memory budget through explicit reservation and release operations. The usable device memory M_{total} is partitioned at startup into:

- M_{fixed}: declared KV cache and activation reserves, measured non-expert model state, safety margin, and a conservative two-expert kernel-conversion workspace per admitted INT4 transition.
- $M^{\text{cap}}_{\text{exp,hi}}$: cap on resident high-precision expert weights.
- $M^{\text{cap}}_{\text{exp,lo}}$: cap on resident low-precision expert weights.
- A global fixed-block staging pool for publish-before-reclaim overlap, so concurrent migration work is allocated and accounted for before it starts.

Every transition must call `try_reserve` before entering the executor. Admission checks the destination-tier cap, the pending-transfer cap, and a cross-tier cap on all committed plus pending expert bytes. The scheduler emits a replacement as an indivisible pair and orders its demotion before its promotion; the global staging pool covers the publish-before-reclaim overlap. When the paired transition frees the destination layer's resident slot, the runtime copies any staging-backed handle into that slot and returns the transient block, preventing staging capacity from becoming long-lived residency. If admission or allocation fails, the request is rejected and reconsidered from the published state at a later tick. Because the device pools are preallocated and admission precedes allocation, concurrent requests cannot grow DynaExQ-owned device memory beyond the startup envelope.

D. Asynchronous Transition Pipeline

The transition pipeline performs data movement and pointer updates while the compute stream runs independently.

a) Versioned-handle publication: The registry maps each expert key to a versioned handle whose published representation fields (tier, block, and packed metadata) are never modified in place; only lease and last-use bookkeeping changes. Publication is a single dictionary replacement under the registry lock. Before issuing expert kernels, an adapter acquires a short read lease on the currently published object and releases it after recording the compute-stream event. A handle retains the latest event for each compute stream, bounding metadata by serving concurrency rather than request count. A monotonically increasing version lets adapters skip unchanged metadata refreshes. During promotion, the destination block is filled on the migration stream and the transition worker waits for its completion event before publishing the new handle. The reclaimer first waits for the old object's lease count to reach zero and then waits for all of its per-stream last-use events. This two-part fence covers the otherwise unsafe interval in which dispatch has read an old handle but has not yet launched its kernel.

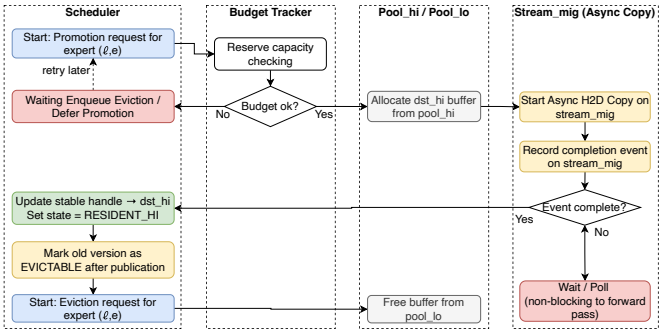


Fig. 5. Promotion and delayed reclamation in the transition pipeline.

b) Stream separation and backpressure: A bounded thread executor issues copies on a dedicated CUDA stream $\texttt{stream}_{\text{mig}}$, disjoint from the default compute stream used by attention and expert kernels. The scheduler keeps each replacement pair intact and submits its demotion first. Backpressure is implemented at admission: a request enters the executor only after passing the budget checks above. A rejected request is not reported as resident; the next decision is recomputed from a snapshot of actually published tiers.

c) Promotion procedure: For a target expert (ℓ, e), the pipeline proceeds as follows (Figure 5):

1) Obtain a successful `try_reserve` for the high-precision footprint.
2) Allocate a destination buffer from `pool_hi`, or from the bounded global staging pool if the resident slot is not yet reclaimable.
3) Issue an asynchronous copy from the pinned host representation on $\texttt{stream}_{\text{mig}}$.
4) Record a CUDA event on $\texttt{stream}_{\text{mig}}$.

The registry publication executes only after the completion event fires, so dispatch never observes a partially populated buffer. The compute stream does not wait on the migration stream; a subsequent dispatch simply resolves the latest published handle.

d) Demotion procedure: When the scheduler marks an expert for demotion, the pipeline publishes the low-precision handle and then reclaims the high-precision buffer. The old handle's read leases close the read-before-launch race; its per-stream compute events then ensure no issued kernel is still reading the buffer before it returns to the pool.

e) Host-side weight preparation: Before timed inference, the prototype materializes both packed tiers in pinned host memory one layer at a time and releases that layer's native source parameters as soon as both versions are complete. This avoids holding the full source model and the full dual-tier cache simultaneously. It next populates every device-resident handle

from the fixed pools. The untimed initialization still increases host DRAM demand but makes a measured transition a copy of an existing representation rather than first-use quantization. The experiment aborts if any expert cannot be cached, released, or registered; it never silently falls back to native weights.

IV. Implementation

DynaExQ is implemented in Python and PyTorch [25] and integrates with Hugging Face Transformers [26]. The implementation is divided into a model-independent control plane and thin model adapters. This separation keeps scheduling and memory-safety logic independent of a particular router or expert MLP layout.

A. Control Plane

a) Observation and scheduling: Model adapters report the router's top-k expert identifiers and routing weights to a `RouterObserver`. A `HotnessTracker` accumulates per-layer counters and updates the EMA state. At a scheduling tick, `PrecisionScheduler` applies the per-layer capacity, hysteresis margin, and transition-rate limit to produce typed promotion and demotion requests. The scheduler never mutates a live expert handle.

The calibration entry point consumes at least 128 uniquely identified prompts from train, validation, development, or explicitly designated calibration splits; test splits are refused. It forces the all-low representation, disables online reassignment, and accumulates each prompt's token-normalized routed probability mass in a separate FP64 counter. Dividing by prompt count yields an order-invariant calibration score, while the FP32 EMA remains dedicated to online adaptation. The entry point emits a full stable ranking per layer and hashes both the source file and selected identifiers.

b) Handles and transition lifecycle: `ExpertRegistry` stores one versioned handle per (ℓ, e) pair. A handle records its precision tier, packed quantization metadata, originating pool block, byte footprint, reservation, active-reader count, and latest event per compute stream. `TransitionEngine` uses a bounded worker pool to execute four stages: obtain the prepared host representation, reserve and allocate the destination block, copy the packed bytes, and publish the new handle. A failed reservation or exhausted pool leaves the previous handle active. After publication, the engine waits for all leases on the old object to drain and then for its per-stream last-use events before returning the block to its originating resident or global staging pool. If an adapter cannot provide a precise event, the implementation conservatively falls back to device synchronization; both paths are counted, and formal asynchronous artifacts require zero global-sync fallbacks.

c) Memory accounting: `BudgetTracker` maintains committed and pending bytes separately for the two precision tiers and enforces a cross-tier live-byte cap. Its `try_reserve`, `commit`, and `release` operations are serialized by a lock, and a staging cap bounds pending work. The allocator pre-allocates disjoint per-layer, per-tier resident pools plus a global transient pool; transitions invoke no global device allocation. A compare-and-swap registry update repatriates staging-backed handles when the paired swap creates a resident hole. Because expert shapes may differ across models, block sizes and counts are derived from the packed representation rather than nominal bit width.

B. Quantized Representations and Dispatch

The quantization layer exposes a common `PackedTensor` abstraction for FP16, group-wise symmetric INT4 (128-weight groups), and group-wise symmetric INT2 (64-weight groups). It stores packed bytes, FP16 scales, tensor dimensions, group size, and the exact storage footprint used by admission control. The reference implementation provides pack, unpack, and dequantize-then-GEMM paths for correctness testing.

A compatibility backend invokes AutoRound's per-tensor symmetric quantizer and converts its output to the same representation; it does not claim AutoRound's full activation-calibrated optimization loop. On CUDA, INT4 dispatch uses PyTorch's fused int4pack matrix multiplication, while INT2 dispatch uses a Triton kernel that unpacks values inside the matrix multiplication and avoids a full FP16 expert temporary.

During migration, the INT4 kernel-native layout replaces the canonical transfer layout in the same pool block; its scale-and-zero overhead is charged to the handle. Conversion briefly creates a nibble-swapped input, native-layout output, and scale/zero tensors before pool rebinding. Startup therefore reserves a conservative two-full-INT4-expert workspace per admitted transition before solving the resident allocation. Scales, native layouts, and conversion workspaces are included in the startup envelope, and each artifact records the selected backend. A paper run aborts if its requested format lacks a budget-safe device kernel.

The Qwen3-MoE, Qwen3-Next, and Phi-3.5-MoE adapters acquire the active handle immediately before routed-expert dispatch and release its lease after the final routed kernel. Multi-matrix experts retain separate metadata for their gate, up, and down projections but share one transition lifecycle. Upon attachment, the Qwen3-MoE adapter selects its handle-aware dispatch instead of Transformers' grouped-expert shortcut, which otherwise bypasses per-expert resolution. Qwen3-Next's shared expert remains on the fixed dense path and is included in the measured dense reservation. An attached adapter fails closed on a missing or incomplete handle; the original model path remains available only before attachment and for equivalence tests.

Timed dynamic experiments first verify that the YAML layer count, routed-expert count, and top-k agree with the checkpoint. They then load the source on CPU, batch-pack both tiers in 16-expert chunks while releasing native sources layerwise, return freed arenas to the OS, and move only the remaining dense model to one specified GPU before allocating pools and registering every routed expert. Startup measures the remaining parameter and buffer bytes, raises an underestimated dense reservation, auto-sizes the conversion workspace, and rejects the run if the device lacks room for the pools plus declared KV, activation, workspace, and safety reserves. Validation also requires the model to both emit routing signals and consume handles.

TABLE II
CONFIGURATION OF EVALUATED MOE MODELS.

	Qwen3-30B	Qwen3-Next-80B	Phi-3.5-MoE
Total Parameters	30.5B	80B	42B
Activated / Token	3.3B	3B	6.6B
Reference Tier	FP16	INT4	FP16
Expert Weight Fraction	95%	96%	96%
Layers	48	48	32
Routed Experts / Layer	128	512 (+1 shared)	16
Top-K	8	10	2

C. Testing and Failure Semantics

CPU tests cover bit packing, quantization error, exact byte accounting, paired rate limiting, reservation races, global staging allocation, publish-after-copy ordering, lease-and-event-fenced reclamation, and repeated transitions without block leakage. Model integration tests compare the Phi-MoE, Qwen3-MoE, and upstream Qwen3-Next adapters with the original forward result when handles point to equivalent weights; the Qwen3-Next test also verifies that source release leaves its shared expert intact. The evaluation CLI distinguishes evaluated-but-incorrect outputs from infrastructure failures. A paired comparison tool verifies identical dataset and sample identities before computing exact McNemar tests with Holm correction.

A separate external-baseline adapter verifies the open-source MoE-Infinity repository revision and recursive source-tree manifest, resolves a pinned model snapshot, reuses the same isolated-model timer, and refuses output without measured prefetch activity and offloaded expert state. Transition artifacts record rejection causes, copied bytes, stage timings, pool occupancy, workspace reservations, and budget counters. During each formal timed generation, a 2 ms NVML poller maps the selected CUDA device to its physical UUID and records current-process GPU memory use; this captures native CUDA-extension and external-runtime allocations omitted by PyTorch's allocator counters. Foreign processes may retain idle GPU memory residency, which is measured separately and excluded from the reported current-process high-water mark, but any nonzero foreign-process NVML utilization invalidates the sample.

V. EVALUATION

We evaluate DynaExQ on a single GPU under a fixed 48 GB device-memory budget. The goal is not minimum per-step latency but a better *quality–throughput–memory* tradeoff within that budget. The experiments ask five questions: (i) does DynaExQ recover accuracy lost to uniform quantization? (ii) how do latency and throughput scale as activation becomes dense? (iii) what does each design component contribute (Sec. V-D)? (iv) does the system stay within its memory budget (Sec. V-E)? (v) how does accuracy change with the high-precision budget (Sec. V-F)?

A. Experimental Setup

Models. We use Qwen/Qwen3-30B-A3B-Instruct-2507 [4], Qwen/Qwen3-Next-80B-A3B-Instruct [18], and microsoft/Phi-3.5-MoE-instruct [19] (Table II).

Benchmarks. We use WikiText-2 [27] for perplexity and MMLU-Pro [28], GPQA [29], AIME25 [30], GSM8K [31], and HumanEval [32] for task-level quality.

Evaluation protocol. We use fixed, identifier-pinned evaluation sets comprising 200 MMLU-Pro examples, all 198 GPQA-Diamond questions, all 30 AIME25 problems, 200 GSM8K examples, and all 164 HumanEval tasks. GPQA answer choices are shuffled once with seed 42, and every method uses the resulting order. Multiple-choice accuracy uses the conditional log-likelihood of each complete answer label rather than a single token; all labels for one question are scored in one unpadded batch so an online precision switch cannot occur between candidates. Numeric answers are taken from an explicit final-answer field, or from the last generated integer when the field is absent; matching an intermediate number does not count. HumanEval reports pass@1 only after executing the official tests with greedy decoding. WikiText perplexity is computed over at most 128 exact 2,048-token windows, with overlapping context labels masked when a shorter stride is used; each artifact stores every window boundary, effective target-token count, mean loss, and NLL so the aggregate can be independently recomputed. We treat the five-task average as a descriptive effect size rather than a pooled significance test. Paired artifacts retain exact McNemar tests with Holm correction as diagnostic companions, but the manuscript makes no pooled significance claim.

Hardware and stack. All experiments run on a single NVIDIA RTX A6000 (48 GB GDDR6 device memory). DynaExQ is integrated into a PyTorch/Transformers-based MoE inference stack [25], [26]. Hot routed experts use a higher-precision tier (FP16 for Qwen3-30B and Phi-3.5-MoE; INT4 for Qwen3-Next-80B); cold routed experts remain at a lower tier (INT4 or INT2, respectively). The Qwen3-Next shared expert remains fixed and is counted with non-routed weights.

Measurement and provenance. Quality runs use greedy decoding with a fixed seed; each binomial accuracy or pass@1 result includes a 95% Wilson interval in its artifact. Latency experiments use exact-length, non-padded 2,048-token inputs and generate 256 tokens per sequence, with five warmup iterations and 100 measured iterations; we report the mean, median, P95, and P99 from the raw samples. Model-level TTFT includes prefill and first-token selection, while TPOT covers subsequent cached decode steps. These measurements exclude request queueing, tokenization, network transport, and response serialization. During each timed generation, NVML samples whole-process GPU memory use on the selected GPU every 2 ms, covering non-PyTorch CUDA allocations; raw PyTorch allocated/reserved high-water counters are retained separately. Every artifact records the checkpoint, method, quantization format, Git commit and dirty state, software versions, GPU model and UUID, seed, raw samples, memory-monitor metadata, and failed-example count. Dataset identity includes repository, configuration, split, immutable repository commit, source row count, and the loader's content fingerprint. A manuscript row is admitted only when all examples were evaluated and its values match the registered JSON artifact.

Baseline selection principle. Static PTQ baselines are chosen so the full expert pool fits in device memory alongside non-

expert weights and runtime allocations, the same constraint DynaExQ operates under. For Qwen3-30B and Phi-3.5-MoE the strongest uniform static point within the single-GPU footprint is INT4. For Qwen3-Next-80B we report two static references: uniform INT4 (higher accuracy, tighter fit) and uniform INT2 (more aggressive compression that leaves headroom for a mixed-precision policy). For an executable offloading comparison, we pin the official MoE-Infinity repository and enable its expert offload, activation-aware cache, and overlapping speculative-prefetch path on Qwen3-30B [7], [33]. The repository states that its current open-source runtime was redesigned and is not identical to the paper implementation; we therefore label this baseline "MoE-Infinity (open source)" and do not attribute its measurements to the original artifact. The pinned runtime does not support the exact Qwen3-Next-80B or Phi-3.5-MoE checkpoints, so we report no fabricated cross-model points.

B. Quality under a Fixed Memory Budget

Table III lists per-benchmark task scores. We interpret results per model because the feasible static baselines differ.

Qwen3-30B. Uniform INT4 drops the average from 67.73 to 65.53. DynaExQ brings it back to 67.22, recovering 77% of the FP16 gap by concentrating higher precision on frequently routed experts while keeping most weights at INT4.

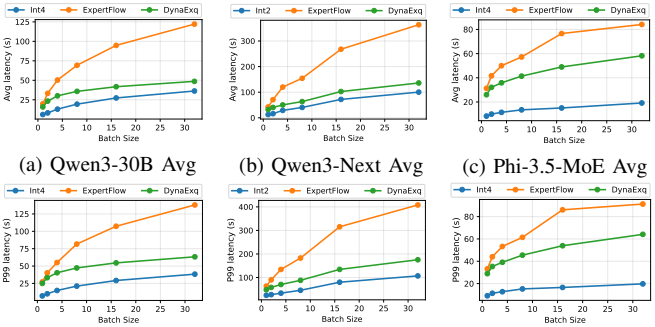
Qwen3-Next-80B. Against uniform INT2 at 73.58, DynaExQ reaches 77.15, recovering 79% of the gap toward INT4 at 78.12. This model shows the largest absolute gain (+3.57 pp) because the INT2 baseline is aggressively compressed, leaving more room for a mixed-precision policy to reclaim quality. We compare against INT2 rather than INT4 because uniform INT4 for Qwen3-Next-80B already fills nearly all 48 GB, leaving no headroom for the mixed-precision hi-pool or for growing the KV cache at larger batch sizes. DynaExQ uses INT2 as the low-precision floor and selectively upgrades the hottest experts to INT4, achieving most of the INT4 accuracy at a fraction of its memory cost.

Phi-3.5-MoE. INT4 lowers the average from 58.02 to 56.13; DynaExQ reaches 57.62, recovering 79% of the gap.

Across all three models, recovery rates fall in the 77–79% range. The degradation and recovery concentrate on reasoning-intensive benchmarks: MMLU-Pro, GPQA, and AIME25. GSM8K and HumanEval change little across precision tiers; this may indicate that their routed working sets are less sensitive to the precision allocations tested here, but the aggregate scores do not identify a causal mechanism.

C. Latency and Throughput

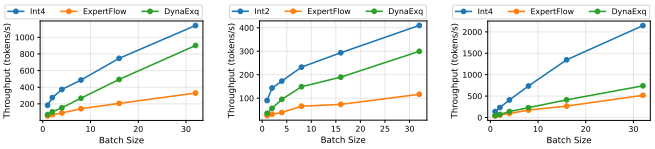
Figure 6 reports model-level average and P99 end-to-end latency. Fully resident static PTQ remains the lower bound because it performs neither online scheduling nor precision transitions; DynaExQ trades some raw speed for adaptation while keeping every routed expert device-resident. On Qwen3-30B, its gap from the pinned MoE-Infinity offload/cache/prefetch runtime widens as batching densifies the routed working set. Because that runtime does not support the other two checkpoints, those panels quantify only DynaExQ's overhead relative to static PTQ. The executable Qwen3-30B result is



(a) Qwen3-30B Avg (b) Qwen3-Next Avg (c) Phi-3.5-MoE Avg

(d) Qwen3-30B P99 (e) Qwen3-Next P99 (f) Phi-3.5-MoE P99

Fig. 6. Model-level end-to-end latency (average and P99) vs. batch size. Qwen3-30B additionally includes the pinned MoE-Infinity open-source runtime; the other checkpoints are unsupported by that version.



(a) Qwen3-30B (b) Qwen3-Next (c) Phi-3.5-MoE

Fig. 7. Generated-token throughput vs. batch size. MoE-Infinity (open source) appears only for its supported Qwen3-30B checkpoint.

checkpoint-specific and does not characterize all offloading systems.

a) Throughput: Figure 7 reports generated tokens/s from the same runs. Static PTQ is the upper reference on all models; DynaExQ preserves monotonic scaling while paying for handle-aware dispatch, scheduling, and background transitions. On Qwen3-30B, MoE-Infinity scales more slowly with batch size, consistent with denser routing exposing more cache-miss movement. Each external-baseline artifact verifies offloaded tensors and measured-interval prefetch calls; invalid configurations are rejected.

D. Component Ablation

We disable one component at a time and measure the effect on Qwen3-30B and Qwen3-Next-80B at batch size 32 (Table IV). All configurations begin from the same independently calibrated ranking prefix and run the same pinned sequence—MMLU-Pro, GPQA-Diamond, AIME25, GSM8K, and HumanEval—followed by five performance warm-ups and 100 measured generations with 2,048 input and 256 output tokens. Accuracy is the unweighted mean over the five task scores; throughput and P99 are measured after this common shift trace. One artifact stores the raw per-example decisions, raw latency samples, active runtime switches, initial-map hash, and the three derived table values. *Static precision* fixes the high-precision map at initialization and disables the online scheduler, similar in spirit to the offline assignment in MxMoE [17]; *Blocking migration* removes the dedicated CUDA migration stream and executes the complete fetch–copy–publish–reclaim transaction inline at each scheduler tick; *w/o Hysteresis* removes the hysteresis margin so that the scheduler re-ranks experts by raw EMA scores every tick.

a) Online scheduling drives accuracy recovery: Static precision freezes the hot set at initialization and cannot track workload shifts. The top hot experts differ between text, math,

TABLE III
TASK-SCORE COMPARISON ACROSS MODELS AND METHODS UNDER COMPARABLE DEVICE-MEMORY BUDGETS.

Model	Method	MMLU-Pro	GPQA	AIME25	GSM8K	HumanEval	Avg.
Qwen3-30B	FP16	76.00	54.04	33.33	90.50	84.76	67.73
	INT4	73.50	50.51	30.00	89.50	84.15	65.53
	DynaExQ	75.00	53.03	33.33	90.00	84.76	**67.22**
Qwen3-Next-80B	INT4	76.00	72.22	70.00	87.00	85.37	78.12
	INT2	71.50	65.66	60.00	86.00	84.76	73.58
	DynaExQ	75.50	71.21	66.67	87.00	85.37	**77.15**
Phi-3.5-MoE	FP16	54.00	36.87	40.00	88.50	70.73	58.02
	INT4	52.00	34.34	36.67	87.50	70.12	56.13
	DynaExQ	53.50	35.86	40.00	88.00	70.73	**57.62**

TABLE IV
COMPONENT ABLATION (BS = 32).

	Qwen3-30B			Qwen3-Next-80B		
Config	Score(%)	Tput(tok/s)	P99(s)	Score(%)	Tput(tok/s)	P99(s)
Full DynaExQ	**67.22**	901	63	**77.15**	297	175
Static precision	66.35	**925**	**58**	75.68	**312**	**162**
Blocking migration	67.12	790	83	77.05	255	235
w/o Hysteresis	67.02	862	71	76.93	279	198

TABLE V
MEMORY UTILIZATION AND RUNTIME OVERHEAD. BATCH SIZE 32 FOR QWEN3-30B AND PHI-3.5-MOE; BATCH SIZE 8 FOR QWEN3-NEXT-80B.

Metric	Qwen3-30B	Qwen3-Next-80B	Phi-3.5-MoE
GPU Memory Budget (GB)	48.0	48.0	48.0
Peak Process GPU Memory (GB)	46.89	43.78	47.05
Resident Expert Pools (GB)	29.71	29.97	21.39
Transient Pool (GB)	0.04	0.01	0.63
Migration Count	536	188	536
Transferred (GB)	3.18	0.24	53.02
Scheduler Mean (ms)	25.25	46.09	2.50
Scheduler P99 (ms)	41.96	72.10	5.67
Pinned Expert Cache (GB)	72.9	61.6	101.29

and code tasks (Fig. 2), so a one-time snapshot mispredicts on later tasks. Accuracy drops by 0.87 pp on Qwen3-30B and 1.47 pp on Qwen3-Next-80B, reducing recovery from 77–79% to 37–46%. The larger drop on Qwen3-Next-80B is consistent with its wider pool of 512 routed experts per layer, for which a fixed high-precision subset covers a smaller fraction of possible identities. Static precision gains marginal throughput (+2.8–4.0%) and lower P99 from zero migration traffic, exposing the performance cost paid for online adaptation.

b) Asynchronous migration preserves throughput and tail latency: Blocking migration retains accuracy, with changes of only -0.04 to -0.06 pp, but causes 12–15% throughput loss and 32–34% P99 inflation. On Qwen3-30B each 9.0 MB FP16 expert copy takes $\sim$0.9 ms. A burst across 10+ layers blocks the compute pipeline for tens of milliseconds. The dedicated migration stream avoids this explicit inline serialization, although it cannot remove shared-bandwidth contention.

c) Hysteresis stabilizes migration traffic: Without hysteresis, boundary experts flip every tick, producing 2–3$\times$ more migration churn. Accuracy changes are small, only -0.12 to -0.23 pp, but throughput drops 4–7% and P99 worsens by 12–13% because the extra transfers compete for PCIe bandwidth.

E. Memory Utilization and Runtime Overhead

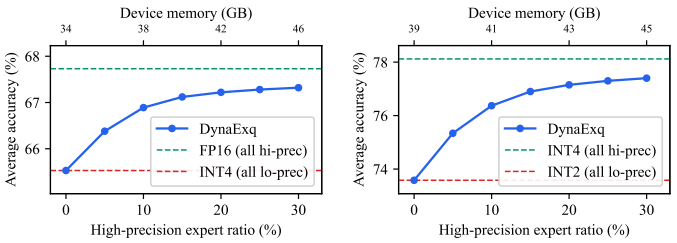
Table V reports completed formal runs for Qwen3-30B and Phi-3.5-MoE at batch size 32 and for Qwen3-Next-80B at batch size 8.

a) Budget compliance: The runtime preallocates fixed resident pools and materializes every routed expert, so conventional "pool utilization" would be approximately 100% by construction and is not an informative empirical metric. We instead report provisioned resident and transient bytes. NVML samples current-process GPU memory every 2 ms; the table reports the maximum across all samples collected during 100 measured generations (Qwen3-30B and Phi-3.5-MoE at batch size 32; Qwen3-Next-80B at batch size 8). Unlike PyTorch allocator counters, this includes allocations owned by CUDA extensions and external runtime components. Other users may retain idle GPU memory residency: their bytes are recorded separately and excluded from our process high-water mark, while any nonzero foreign-process NVML utilization invalidates the sample. A current-process high-water mark above the declared 48 GB device-memory envelope also invalidates the run. Raw PyTorch allocated/reserved counters are retained as diagnostic companions, and the final budget snapshot separately proves that committed plus pending DynaExQ-owned expert bytes never exceeded their cap.

b) Migration traffic: Migration count sums completed promotions and demotions over five warm-ups and 100 measured generations. Transferred bytes are accumulated from the exact packed payload copied for every completed transition; no nominal per-expert size is multiplied by a request count. These counters make workload-dependent churn auditable without inferring traffic from the final precision map.

c) Control-plane cost: For every successful scheduling tick, the wrapper records the CPU time required to synchronize the published tier map, rank candidates, and enqueue accepted requests. The table reports the mean and nearest-rank P99 from those raw samples. Under normal asynchronous operation this excludes background fetch, copy, and reclamation, which are reported separately through transition-stage timings.

d) Host DRAM tradeoff: The pinned expert cache is the system's main auxiliary cost. We report the byte sum of both canonical packed representations, including scales, rather than process RSS. Native source tensors are released after packing, but allocator metadata and temporary host buffers



(a) Qwen3-30B (b) Qwen3-Next-80B

Fig. 8. Average accuracy vs. high-precision expert budget (n_{hi}/E). The dashed line marks the static all-low baseline at 0%.

can make peak process memory larger than this payload-only value. Keeping both versions makes promotion a direct copy with no online quantization, and the host-side footprint does not compete with the device-memory budget.

F. Precision Budget Sensitivity

Figure 8 sweeps the requested per-layer high-precision ratio n_{hi}/E from 0% (all low-precision, equivalent to the static PTQ baseline) to 30% on the five accuracy benchmarks. For a requested ratio r, the runtime allocates exactly $\lfloor rE \rfloor$ high-precision slots in every layer, records the realized ratio after integer rounding, and rejects rather than silently shrinking an infeasible point. Every point uses the ablation protocol's pinned task order and registered benchmark subsets. WikiText-2 is excluded because its metric is perplexity.

Both curves are concave. This pattern is consistent with heavy-tailed routing: a small fraction of hot experts absorbs much of the token traffic, so upgrading that subset can yield disproportionate accuracy gains.

On Qwen3-30B the first 5% of high-precision experts recovers 0.85 pp of the 2.20 pp FP16–INT4 gap, while the next 5% adds only 0.51 pp. Each upgrade adds 6.68 MiB per expert under the packed layout, so the memory price of the first 5% is already non-trivial. On Qwen3-Next-80B the effect is more pronounced: the first 5% recovers 1.76 pp of the 4.54 pp INT4–INT2 gap, because the wider precision gap amplifies per-expert quality impact while each upgrade adds only 0.703 MiB. Beyond 25%, gains on both models fall below 0.1 pp per 5% increment. We stop the sweep at 30% because Qwen3-30B already reaches 46.2 GB at that point, leaving less than 2 GB of headroom before the 48 GB GPU memory ceiling.

DynaExQ's default at ∼20% sits near the knee of both curves, recovering 77–79% of the respective accuracy gap at 42–43 GB of the 48 GB budget. The knee point falls at the same ratio under both FP16/INT4 and INT4/INT2 regimes, which suggests the EMA-based scheduler captures routing concentration at similar efficiency regardless of precision pair. For deployment, this curve provides a concrete tradeoff map: shifting from the default 20% to 10% costs ∼0.33 pp on Qwen3-30B while recovering roughly 4 GB of GPU memory—headroom that translates directly into larger batch sizes or longer KV caches.

VI. Related Work

We organize prior work by the assumption each line makes about expert weights at inference time. The observation that high-utilization experts justify higher precision appears across MoE quantization and hybrid offload work. We therefore do not claim runtime importance scoring or mixed precision in isolation. The distinction of DynaExQ is its fully resident, single-GPU execution model: one complete version of every routed expert remains addressable while versioned publication, admission-before-allocation, and deterministic resident/staging pools make online precision replacement auditable under a declared device-memory envelope.

TABLE VI
Mechanism boundary relative to representative baselines. For a partial device cache, serving an expert absent from the GPU requires a host-to-device transfer, although prefetch may hide some or all of its latency.

System	Device expert state	Online control target	Host transfer on cache miss
Uniform PTQ	Complete, fixed	None (offline map)	N/A (fully resident)
MoE-Infinity [7]	Partial GPU cache	Placement/prefetch	Required
HOBBIT [11]	Partial GPU cache	Miss precision + placement	Required
ExpertFlow [13]	Partial GPU cache	Prefetch/routing	Required
DynaExQ	Complete, versioned	Precision residency	N/A (fully resident)

A. MoE Offloading and Memory Hierarchy Management

General-purpose LLM inference engines such as vLLM [24], SGLang [34], and FlexGen [35] manage KV-cache memory, request scheduling, or single-GPU offloading but do not address MoE-specific expert residency. A complementary line treats GPU memory as a cache: Adaptive gating [5], EdgeMoE [6], MoE-Infinity [7], Mixtral-offloading [8], Pre-gated MoE [9], AdaMoE [10], ProMoE [12], and ExpertFlow [13] differ in prediction, staging, and coordination but primarily make *placement* and *fetch time* the control variables (Table VI). MoE-Infinity, for instance, builds a locality-aware offload cache but has no notion of versioned precision residency. ProMoE schedules proactive prefetching for fixed-precision weights, while ExpertFlow adapts its prediction horizon and coordinates an offload cache. These systems optimize which absent expert to fetch; DynaExQ instead keeps all routed experts resident and changes the precision of already-addressable experts.

B. Static PTQ and Generic Low-Bit Inference

Weight-only and activation-aware PTQ schemes such as GPTQ [15], AWQ [14], and LLM.int8() [22] improve accuracy at aggressive bit widths but produce a *static* precision configuration. Even methods that adapt bit choices at calibration time deploy a fixed precision map at inference. Theoretically grounded expert-wise allocation [36] further improves static assignments using router change and quantization-noise proxies. These methods are complementary sources of offline sensitivity priors; they do not define a concurrent online replacement protocol.

C. MoE-Specific Mixed Precision

MxMoE [17] derives mixed-precision configurations from expert sensitivity and couples them with mixed-precision grouped GEMMs. MoPEQ [37] assigns bit widths using activation frequency and sensitivity measures. Earlier work on routing stability [38] and expert specialization [39] explored

how expert heterogeneity affects model quality, providing motivation for non-uniform precision allocation. These methods improve the quality–memory tradeoff at model-build time. MxMoE, for example, produces mixed-precision kernels but assigns bit widths statically with no runtime residency control. The resulting precision map stays *fixed* during inference and does not adapt as routing distributions shift.

D. Hybrid Systems

HOBBIT [11] combines mixed-precision expert representations with offloading to reduce the load cost on cache-miss paths, showing that precision heterogeneity and memory hierarchy policies interact. DyMoE [40] is the closest recent system: it dynamically classifies expert importance, applies depth-aware 8/4/0-bit policies, and prefetches missing expert versions from CPU or SSD for 12–24 GB edge devices. DynaExQ studies a different operating point and invariant. It maintains one complete device-resident version of every routed expert on a 48 GB GPU, uses long-horizon routing statistics rather than per-phase look-ahead, and focuses on safe replacement of resident representations through read leases, last-use events, publish-after-complete handles, and byte-level admission control. It neither skips experts nor claims to optimize offloading latency. Thus DyMoE's depth/phase-aware policy and DynaExQ's replacement mechanisms are potentially complementary; a direct comparison on common hardware remains future work.

VII. Limitations and Scope

DynaExQ has five principal limitations. First, it keeps kernel-ready high- and low-precision expert representations in host memory. This removes online repacking from the critical path but increases pinned-DRAM demand and may be unsuitable for edge hosts with limited memory. Disk-backed preparation would reduce DRAM use at the cost of another latency tier.

Second, the current policy optimizes aggregate routing utility. It does not provide per-tenant quality guarantees or fairness when concurrent requests have different domains. Per-request precision decisions would also make batching less regular. Extending the objective with tenant weights or service-level constraints is therefore a policy problem that the present mechanism does not solve.

Third, our evaluation covers one 48 GB GPU and three model families. Transfer/compute overlap and the feasible high-precision capacity depend on the GPU, PCIe generation, sequence length, and KV-cache policy, so the operating points are platform measurements. The pinned MoE-Infinity open-source runtime supports Qwen3-30B but not the other two checkpoints, and it differs from the implementation in its paper. The design uses two precomputed tiers on one GPU and a fixed task order.

Fourth, the prototype favors correctness and inspectability over kernel maturity. Handle-aware dispatch currently uses eager expert execution for Qwen-family models because the upstream grouped-GEMM shortcut does not accept per-expert versioned handles. This isolates the systems effect under study but leaves integration with a fused handle-aware grouped kernel to future work. Dual-tier packing and pinned-host-cache construction are untimed startup costs; production deployments would need to amortize or persist this preparation.

Finally, the hard invariant applies to DynaExQ-owned expert pools, staging blocks, and declared conversion workspace. End-to-end device-memory safety is conditional on the serving configuration keeping KV-cache, activation, and framework allocations within their startup reservations; the prototype does not replace or police the framework allocator. A separate CUDA stream removes explicit migration synchronization from the forward path but cannot eliminate resource contention: copies and repacking kernels may still share memory-system bandwidth with inference. Our latency scope is an isolated model and excludes request queueing, tokenization, network transport, and serialization. We therefore compare model-level end-to-end and tail latency, not only scheduler CPU time.

VIII. Conclusion

DynaExQ manages expert precision online within an explicit memory envelope. Routing-driven allocation, versioned handles, admission control, and fixed-block pools replace representations safely. The system recovers much of the quality lost to uniform low-bit quantization while keeping all routed experts device-resident. Its invariant covers runtime-owned expert and transition memory; end-to-end safety also requires the serving stack's KV-cache, activation, and framework reservations. Future work should reduce host duplication and extend quality isolation and budget coordination to multi-tenant, multi-GPU serving.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[3] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[4] Q. Team, "Qwen3 technical report," 2025.

[5] J. Li, Q. Su, Y. Yang, Y. Jiang, C. Wang, and H. Xu, "Adaptive gating in mixture-of-experts based language models," *arXiv preprint arXiv:2310.07188*, 2023.

[6] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, "EdgeMoE: Fast on-device inference of MoE-based large language models," *arXiv preprint arXiv:2308.14352*, 2023.

[7] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Offloading-efficient MoE model serving," *arXiv preprint arXiv:2401.14361*, 2024.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[9] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1018–1031.

[10] S. Zhong, L. Liang, Y. Wang, R. Wang, R. Huang, and M. Li, "AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.

[11] P. Tang, J. Liu, X. Hou, Y. Pu, J. Wang, P.-A. Heng, C. Li, and M. Guo, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433*, 2024.

[12] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[13] Z. Shen, K. Chu, Y. Zhang, D. Xiang, R. Wu, and W. Zhang, "ExpertFlow: Adaptive expert scheduling and memory coordination for efficient MoE inference," arXiv preprint arXiv:2510.26730, 2025.

[14] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.

[15] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[16] Y. J. Kim, R. Fahim, and H. H. Awadalla, "Mixture of quantized experts (MoQE): Complementary effect of low-bit quantization and robustness," arXiv preprint arXiv:2310.02410, 2023.

[17] H. Duanmu, X. Li, Z. Yuan, S. Zheng, J. Duan, X. Zhang, and D. Lin, "MxMoE: Mixed-precision quantization for MoE with accuracy and performance co-design," arXiv preprint arXiv:2505.05799, 2025.

[18] Qwen Team, "Qwen3-Next-80B-A3B-Instruct model card," https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct, 2025, accessed: 2026-07-29.

[19] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," 2024.

[20] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "Swapmoe: Serving off-the-shelf moe-based large language models with tunable memory budget," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 6710–6720.

[21] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International conference on machine learning*. PMLR, 2023, pp. 38 087–38 099.

[22] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 30 318–30 332.

[23] J. Zhang, Y. Zhang, B. Zhang, Z. Liu, and D. Cheng, "MoQE: Improve quantization model performance via mixture of quantization experts," *arXiv preprint arXiv:2508.09204*, 2025.

[24] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.

[25] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "PyTorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[26] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.

[27] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," 2016.

[28] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang *et al.*, "MMLU-Pro: A more robust and challenging multi-task language understanding benchmark," *Advances in Neural Information Processing Systems*, vol. 37, pp. 95 266–95 290, 2024.

[29] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman, "GPQA: A graduate-level google-proof q&a benchmark," in *First Conference on Language Modeling*, 2024.

[30] Y. Zhang and M.-A. Team, "American invitational mathematics examination (aime) 2025," Hugging Face dataset, 2025. [Online]. Available: https://huggingface.co/datasets/math-ai/aime25

[31] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[32] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan *et al.*, "Evaluating large language models trained on code," 2021.

[33] EfficientMoE Team, "MoE-Infinity official open-source runtime," https://github.com/EfficientMoE/MoE-Infinity, 2026, accessed 2026-07-29.

[34] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kober, L. Shi, C. Wu *et al.*, "SGLang: Efficient execution of structured language model programs," *arXiv preprint arXiv:2312.07104*, 2024.

[35] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 1–16, 2023.

[36] M. N. R. Chowdhury, K. E. Maghraoui, H. Tsai, N. Wang, G. W. Burr, L. Liu, and M. Wang, "Efficient quantization of mixture-of-experts with theoretical generalization guarantees," in *The Fourteenth International Conference on Learning Representations*, 2026.

[37] K. T. Chitty-Venkata, J. Ye, and M. Emani, "MoPEQ: Mixture of mixed precision quantized experts," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 4023–4032.

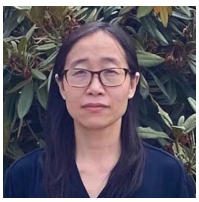
[38] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[39] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.

[40] Y. Huang, Z. Fang, W. Luo, R. Wu, W. Chen, and Z. Zheng, "DyMoE: Dynamic expert orchestration with mixed-precision quantization for efficient MoE inference on edge," arXiv preprint arXiv:2603.19172, 2026.

Kexin Chu Kexin Chu received the B.S. and M.S. degrees in computer science from Hefei University of Technology, Hefei, China, in 2017 and 2020, respectively. He is currently working toward the Ph.D. degree in computer science with the School of Computing, University of Connecticut, Storrs, CT, USA. His research interests include machine learning systems, large language model serving and infrastructure, KV-cache and memory-system optimization, and trustworthy AI infrastructure. Before beginning his doctoral studies, he was a backend software engineer at Baidu, where he designed and maintained large-scale production services, and he interned on model deployment and inference optimization. His recent work focuses on efficient and reliable LLM inference under realistic device-memory constraints, including online precision residency for Mixture-of-Experts models. Contact him at kexin.chu@uconn.edu.

Dawei Xiang Dawei Xiang received the M.S. degree in computer science and engineering from Texas A&M University, College Station, TX, USA, in 2024. He is currently working toward the Ph.D. degree in computer science with the School of Computing, University of Connecticut, Storrs, CT, USA, where he is also a research assistant. His research interests include machine learning systems, large language model inference, agentic workflows, and model optimization under deployment constraints. He has worked on efficient serving and runtime mechanisms for modern language models, with an emphasis on Mixture-of-Experts execution when the full expert pool cannot remain in high precision on a single GPU. Contact him at ieb24002@uconn.edu.



Wei Zhang Wei Zhang received the Ph.D. degree in computer architecture from Beihang University, Beijing, China, in 2015, and the Ph.D. degree in computer science from The George Washington University, Washington, DC, USA, in 2018. She is an Assistant Professor with the School of Computing, University of Connecticut, Storrs, CT, USA. Her research interests include cloud computing and virtualization, distributed systems, operating systems, resource disaggregation, networked and storage systems, serverless computing, and machine learning systems. Contact her at wei.13.zhang@uconn.edu.